

```

1 import torch
2 torch.manual_seed(42)
3 from transformers import AutoTokenizer, AutoModelForCausalLM
4
5 tokenizer = AutoTokenizer.from_pretrained('Qwen/Qwen2-0.5B', dtype=torch.float32)
6 model = AutoModelForCausalLM.from_pretrained('Qwen/Qwen2-0.5B', dtype=torch.float32)
7
8 messages = ["The Eiffel Tower is located in"]
9 ]
10
11 inputs = tokenizer(messages, return_tensors='pt')
12
13 outputs = model(**inputs)
14 print(outputs.logits.shape)
15

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```

warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
config.json: 100%                               661/661 [00:00<00:00, 66.7kB/s]

tokenizer_config.json: 1.29k/? [00:00<00:00, 47.7kB/s]

vocab.json: 2.78M/? [00:00<00:00, 44.8MB/s]

merges.txt: 1.67M/? [00:00<00:00, 44.3MB/s]

tokenizer.json: 7.03M/? [00:00<00:00, 80.0MB/s]

model.safetensors: 100%                               988M/988M [00:17<00:00, 58.6MB/s]

Loading weights: 100%                               290/290 [00:02<00:00, 132.81it/s, Materializing param=model.norm.weight]

generation_config.json: 100%                         138/138 [00:00<00:00, 6.23kB/s]

torch.Size([1, 8, 151936])

```

```

1 from google.colab import drive
2 drive.mount('/content/drive')

```

Mounted at /content/drive

torch.Size([1, 26, 151936])-->(B, N_TOKENS, VOCAB_SIZE)

```

1 last_token = outputs.logits[0, -1,:]
2 token_id = torch.argmax(last_token)
3 decode = tokenizer.decode(token_id)
4 print(f"Token id: {token_id}")
5 print(f"Decoded to text: {decode}")

```

Token id: 12095
Decoded to text: Paris

```
1 tokenizer.decode(torch.argmax(outputs.logits, dim=-1))
```

[' following-el Tower is the in Paris']

```
1 print(model)
```

```

Qwen2ForCausalLM(
  (model): Qwen2Model(
    (embed_tokens): Embedding(151936, 896)
    (layers): ModuleList(
      (0-23): 24 x Qwen2DecoderLayer(
        (self_attn): Qwen2Attention(
          (q_proj): Linear(in_features=896, out_features=896, bias=True)
          (k_proj): Linear(in_features=896, out_features=128, bias=True)
          (v_proj): Linear(in_features=896, out_features=128, bias=True)
          (o_proj): Linear(in_features=896, out_features=896, bias=False)
        )
      )
    (mlp): Qwen2MLP(

```

```

        (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
        (up_proj): Linear(in_features=896, out_features=4864, bias=False)
        (down_proj): Linear(in_features=4864, out_features=896, bias=False)
        (act_fn): SiLUActivation()
    )
    (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
    (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
)
)
(norm): Qwen2RMSNorm((896,), eps=1e-06)
(rotary_emb): Qwen2RotaryEmbedding()
)
(lm_head): Linear(in_features=896, out_features=151936, bias=False)
)

```

```
1 model.model.layers[12].mlp
```

```

Qwen2MLP(
  (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
  (up_proj): Linear(in_features=896, out_features=4864, bias=False)
  (down_proj): Linear(in_features=4864, out_features=896, bias=False)
  (act_fn): SiLUActivation()
)

```

```

1 captured = {}
2 layers = [9, 10, 11, 12, 13, 14]
3
4 def get_hook(layer_id):
5     def hook(module, input, output):
6         captured[layer_id] = output.cpu().detach()
7     return hook
8
9 handles = [model.model.layers[n].mlp.register_forward_hook(get_hook(n)) for n in layers]
10

```

```

1 from datasets import load_dataset
2 ds = load_dataset("wikitext", "wikitext-103-raw-v1", split="train[:500]")

```

```

README.md:      10.5k/? [00:00<00:00, 798kB/s]

wikitext-103-raw-v1/test-00000-of-00001(...): 100%          733k/733k [00:00<00:00, 1.96MB/s]

wikitext-103-raw-v1/train-00000-of-00002(...): 100%         157M/157M [00:03<00:00, 115MB/s]

wikitext-103-raw-v1/train-00001-of-00002(...): 100%         157M/157M [00:02<00:00, 121MB/s]

wikitext-103-raw-v1/validation-00000-of-...): 100%         657k/657k [00:00<00:00, 2.86MB/s]

Generating test split: 100%                                4358/4358 [00:00<00:00, 52613.97 examples/s]

Generating train split: 100%                               1801350/1801350 [00:06<00:00, 73135.81 examples/s]

Generating validation split: 100%                          3760/3760 [00:00<00:00, 119150.06 examples/s]

```

```
1
```

```

1 text = [text for text in ds['text'] if text.strip()]
2 len(text)

```

```
338
```

```

1 store= {9: [],
2         10: [],
3         11: [],
4         12: [],
5         13: [],
6         14 :[]
7         }
8
9 for i in text:
10     tokenids = tokenizer(i,
11                          return_tensors='pt',
12                          truncation=True,
13                          max_length=128)
14     with torch.no_grad():
15         y_pred = model(**tokenids)
16

```

```

17 for layer in store:
18     store[layer].append(captured[layer])
19
20 for handle in handles:
21     handle.remove()

```

```
1 len(store[12])
```

```
338
```

```
1 store[12][0].shape
```

```
torch.Size([1, 8, 896])
```

```

1 activation_9 = torch.cat([t.squeeze(0) for t in store[9]], dim=0)
2 activation_10 = torch.cat([t.squeeze(0) for t in store[10]], dim=0)
3 activation_11 = torch.cat([t.squeeze(0) for t in store[11]], dim=0)
4 activation_12 = torch.cat([t.squeeze(0) for t in store[12]], dim=0)
5 activation_13 = torch.cat([t.squeeze(0) for t in store[13]], dim=0)
6 activation_14 = torch.cat([t.squeeze(0) for t in store[14]], dim=0)

```

```
1 activation_10.shape
```

```
torch.Size([18639, 896])
```

```

1 from torch import nn
2
3 class SAE(nn.Module):
4
5     def __init__(self, in_dim, out_dim):
6         super().__init__()
7         self.encoder = nn.Linear(in_features=in_dim, out_features=out_dim)
8         self.decoder = nn.Linear(in_features=out_dim, out_features=in_dim)
9         self.relu = nn.ReLU()
10
11     def forward(self, x: torch.Tensor):
12
13         hidden = self.relu(self.encoder(x))
14         reconstruction = self.decoder(hidden)
15         return reconstruction, hidden

```

```

1 sparsity = SAE(in_dim=896, out_dim=3584)
2 sparsity

```

```

SAE(
  (encoder): Linear(in_features=896, out_features=3584, bias=True)
  (decoder): Linear(in_features=3584, out_features=896, bias=True)
  (relu): ReLU()
)

```

```
1 sparsity(activation_14)
```

```

(tensor([[ 0.0660,  0.0098, -0.0375, ...,  0.0160, -0.0016,  0.0094],
         [ 0.0293, -0.0201, -0.0650, ...,  0.0374,  0.0265,  0.0016],
         [-0.0116, -0.0309, -0.0358, ...,  0.0357,  0.0018,  0.0081],
         ...,
         [ 0.0286,  0.0035, -0.0504, ...,  0.0004,  0.0503, -0.0527],
         [ 0.0391, -0.0018, -0.0497, ...,  0.0228,  0.0200,  0.0104],
         [-0.0048,  0.0226, -0.0396, ...,  0.0734,  0.0272,  0.0107]]),
 grad_fn=<AddmmBackward0>),
 tensor([[0.0000, 0.0000, 0.0253, ..., 0.1533, 0.0000, 0.1134],
         [0.0473, 0.0170, 0.0653, ..., 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.1462, ..., 0.0000, 0.1031, 0.0000],
         ...,
         [0.0443, 0.0000, 0.0000, ..., 0.0000, 0.0000, 0.0000],
         [0.0821, 0.0884, 0.0000, ..., 0.0000, 0.0000, 0.0337],
         [0.0000, 0.0920, 0.0587, ..., 0.0000, 0.0000, 0.0000]],
 grad_fn=<ReluBackward0>))

```

```
1 from torch.utils.data import DataLoader
```

```
1 train_dataloader = DataLoader(activation_14, batch_size=256)
2 tok = next(iter(train_dataloader))
3 tok.shape
```

```
torch.Size([256, 896])
```

```
1 tok.shape
```

```
torch.Size([256, 896])
```

```
1 loss_fn = nn.MSELoss()
2 optimizer = torch.optim.Adam(sparsity.parameters(), lr=1e-4)
3 lambda_val = 5e-1
4 for epoch in range(55):
5     total_loss = 0
6     for x in train_dataloader:
7         optimizer.zero_grad()
8         reconstruction, hidden = sparsity(x)
9         loss = loss_fn(reconstruction, x) + lambda_val * hidden.abs().mean()
10        loss.backward()
11        optimizer.step()
12        total_loss+=loss.item()
13    print(f"Epoch {epoch+1} loss: {total_loss / len(train_dataloader):.4f}")
```

```
Epoch 1 loss: 0.0489
Epoch 2 loss: 0.0385
Epoch 3 loss: 0.0350
Epoch 4 loss: 0.0322
Epoch 5 loss: 0.0300
Epoch 6 loss: 0.0281
Epoch 7 loss: 0.0264
Epoch 8 loss: 0.0250
Epoch 9 loss: 0.0236
Epoch 10 loss: 0.0225
Epoch 11 loss: 0.0215
Epoch 12 loss: 0.0206
Epoch 13 loss: 0.0197
Epoch 14 loss: 0.0190
Epoch 15 loss: 0.0183
Epoch 16 loss: 0.0177
Epoch 17 loss: 0.0172
Epoch 18 loss: 0.0167
Epoch 19 loss: 0.0162
Epoch 20 loss: 0.0158
Epoch 21 loss: 0.0154
Epoch 22 loss: 0.0151
Epoch 23 loss: 0.0148
Epoch 24 loss: 0.0144
Epoch 25 loss: 0.0142
Epoch 26 loss: 0.0139
Epoch 27 loss: 0.0136
Epoch 28 loss: 0.0134
Epoch 29 loss: 0.0131
Epoch 30 loss: 0.0129
Epoch 31 loss: 0.0127
Epoch 32 loss: 0.0125
Epoch 33 loss: 0.0123
Epoch 34 loss: 0.0121
Epoch 35 loss: 0.0119
Epoch 36 loss: 0.0117
Epoch 37 loss: 0.0116
Epoch 38 loss: 0.0114
Epoch 39 loss: 0.0113
Epoch 40 loss: 0.0111
Epoch 41 loss: 0.0110
Epoch 42 loss: 0.0108
Epoch 43 loss: 0.0107
Epoch 44 loss: 0.0106
Epoch 45 loss: 0.0105
Epoch 46 loss: 0.0103
Epoch 47 loss: 0.0102
Epoch 48 loss: 0.0101
Epoch 49 loss: 0.0100
Epoch 50 loss: 0.0099
Epoch 51 loss: 0.0098
Epoch 52 loss: 0.0097
Epoch 53 loss: 0.0096
Epoch 54 loss: 0.0095
Epoch 55 loss: 0.0094
```

```

1
2 with torch.no_grad():
3     batch = next(iter(train_dataloader))
4     _, hidden = sparsity(batch)
5     sparsity_ratio = (hidden == 0).float().mean()
6     print(f"Sparsity: {sparsity_ratio:.4f}")

```

Sparsity: 0.8665

```

1 with torch.no_grad():
2     _, hidden = sparsity(activation_14)
3     n_0 = hidden[:, 0]
4     top10 = torch.topk(n_0, 10)
5     print(top10.indices)

```

tensor([9858, 9916, 9932, 9842, 11861, 9742, 9821, 9872, 9498, 9802])

```

1 sum(t.shape[1] for t in store[14])

```

18639

```

1 lengths = torch.tensor([t.shape[1] for t in store[14]])

```

```

1 lengths

```

```

tensor([ 8, 128, 110, 116, 6, 128, 128, 91, 6, 128, 128, 128, 6, 128,
        128, 8, 128, 8, 128, 107, 6, 86, 128, 128, 103, 6, 80, 9,
        128, 128, 128, 10, 128, 102, 6, 128, 7, 128, 96, 18, 128, 128,
        48, 17, 23, 23, 86, 93, 88, 23, 20, 20, 21, 12, 16, 11,
        10, 18, 11, 128, 99, 72, 128, 128, 128, 128, 106, 96, 64, 26,
        16, 17, 9, 17, 16, 7, 4, 21, 13, 6, 24, 6, 6, 4,
        9, 101, 128, 46, 28, 8, 128, 128, 10, 126, 68, 7, 128, 56,
        128, 128, 7, 109, 94, 99, 6, 9, 99, 84, 13, 65, 99, 92,
        19, 109, 110, 10, 128, 128, 9, 45, 95, 128, 11, 108, 86, 6,
        87, 91, 83, 11, 128, 13, 128, 128, 6, 8, 17, 17, 20, 20,
        20, 15, 17, 22, 27, 13, 8, 16, 14, 16, 25, 16, 14, 17,
        14, 16, 27, 24, 17, 15, 14, 17, 22, 19, 17, 16, 17, 23,
        16, 22, 17, 18, 14, 15, 14, 15, 17, 13, 16, 13, 19, 21,
        16, 14, 27, 17, 16, 9, 20, 23, 9, 14, 20, 19, 28, 23,
        21, 21, 35, 25, 14, 23, 30, 11, 128, 7, 128, 128, 59, 8,
        82, 128, 6, 128, 128, 11, 128, 128, 6, 128, 128, 8, 128, 8,
        101, 128, 7, 128, 16, 128, 128, 92, 10, 128, 128, 7, 10, 128,
        128, 10, 128, 128, 128, 128, 10, 128, 128, 50, 10, 114, 93, 7,
        79, 15, 24, 8, 12, 9, 11, 12, 14, 7, 96, 9, 4, 6,
        6, 6, 6, 7, 11, 10, 4, 8, 9, 6, 5, 9, 17, 7,
        8, 34, 7, 128, 6, 128, 6, 128, 7, 128, 7, 128, 7, 128,
        7, 128, 6, 102, 128, 59, 11, 128, 8, 128, 6, 128, 7, 128,
        7, 128, 7, 23, 17, 18, 17, 6, 8, 128, 7, 128, 6, 128,
        7, 12, 128, 12, 128, 128, 128, 7, 128, 128, 128, 128, 128,
        113, 128])

```

```

1 boundaries = torch.cumsum(lengths, dim=0)
2 print(boundaries[:10])

```

tensor([8, 136, 246, 362, 368, 496, 624, 715, 721, 849])

```

1 top10

```

```

torch.return_types.topk(
  values=tensor([0.9890, 0.9625, 0.9448, 0.9175, 0.9153, 0.8634, 0.8336, 0.8246, 0.8088,
                0.7574]),
  indices=tensor([ 9858, 9916, 9932, 9842, 11861, 9742, 9821, 9872, 9498, 9802]))

```

```

1 index = []
2 for i in top10.indices:
3     index.append((boundaries>i).nonzero()[0])
4 index

```

```

[tensor([182]),
 tensor([185]),
 tensor([186]),
 tensor([181]),
 tensor([222]),
 tensor([175]),
 tensor([180]),
 tensor([183]),

```

```
tensor([161]),
tensor([179])]
```

```
1 sd_idx = []
2
3 _, top10_ = hidden.topk(10, dim=-1)
4
5 top10_act = [w.tolist() for w in top10_]
6
7
8 for i in top10_:
9
10 indices = [(boundaries>x).nonzero()[0].item() for x in i]
11
12 sd_idx.append(tuple(n for n in indices))
```

```
1 len(top10_act)
```

18639

```
1
2 top10_[-1, :]
```

tensor([1892, 262, 3142, 1769, 2359, 321, 2555, 2293, 372, 1934])

```
1 xxd = [w.tolist() for w in top10_]
```

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'sentence index': sd_idx,
5     'feature indices': top10_act
6 })
7 df.head()
```

	sentence index	feature indices	
0	(30, 13, 41, 1, 41, 28, 14, 21, 41, 41)	[2659, 1158, 3486, 45, 3549, 2414, 1289, 1778, ...	
1	(37, 5, 26, 33, 14, 31, 19, 1, 30, 41)	[3086, 390, 2249, 2864, 1304, 2682, 1743, 62, ...	
2	(3, 10, 24, 41, 7, 30, 21, 21, 21, 13)	[305, 960, 2112, 3485, 707, 2601, 1752, 1777, ...	
3	(35, 1, 35, 7, 7, 10, 14, 21, 13, 29)	[2941, 12, 2939, 707, 650, 974, 1304, 1837, 11...	
4	(1, 5, 26, 22, 16, 1, 1, 25, 38, 6)	[77, 376, 2220, 1839, 1492, 57, 12, 2202, 3255...	

Next steps: [New interactive sheet](#)

```
1 feature_one_interpretability = pd.DataFrame({
2     'sentence index per neuron': [i for i in df['sentence index'][0]],
3     'feature indices per neuron': [i for i in df['feature indices'][0]],
4     'sentence': [text[i] for i in df['sentence index'][0]]
5 })
```

```
1 feature_one_interpretability.head()
```

	sentence index per neuron	feature indices per neuron	sentence
0	30	2659	Two manga adaptations were produced , followi...
1	13	1158	Concept work for Valkyria Chronicles III bega...
2	41	3486	This movement is prompted by the feeling that...
3	1	45	Senjō no Valkyria 3 : Unrecorded Chronicles (...)
4	41	3549	This movement is prompted by the feeling that...

Next steps: [New interactive sheet](#)

```
1 hidden[:, 2659].topk(10)
```

1

1_dec

```
1 feature_one_interpretability.head()
```

Next steps: ([New interactive sheet](#))

save successfully

7/10

```
14 ""
15 Neuron 2659 (token: "1", sentence: Valkyria manga adaptations)
16 "Capitalized poetic title words in Japanese media adaptations"
17 Neuron 1158 (token: ",", sentence: Valkyria Chronicles III development)
18 "Comma punctuation in dense multi-clause game development prose"
19 Neuron 3486 (token: "aspect", sentence: Little Rock Arsenal speech)
20 "Formal abstract nouns in 19th century legal authority language"
21 Neuron 45 (token: ") ", sentence: Valkyria Chronicles III intro)
22 "Closing parentheses in bilingual Japanese-English title translations"
23 Neuron 3549 (token: "charge", sentence: Little Rock Arsenal speech)
24 "Words of authority and custody in formal governmental demand language"
25 Neuron 2414 (token: "period", sentence: Valkyria Chronicles anime)
26 "Time period references in entertainment media release scheduling language"
27 Neuron 1289 (token: "mission", sentence: Valkyria Chronicles III development)
28 "Military mission terminology in video game design and development context"
29 Neuron 1778 (token: "sales", sentence: Valkyria Chronicles III sales)
30 "Sales and commercial performance metrics in Japanese gaming charts"
31 Neuron 3551 (token: "the", sentence: Little Rock Arsenal speech)
32 "Definite article 'the' in formal governmental possession and authority claims"
33 Neuron 3438 (token: "citizens", sentence: Little Rock Arsenal speech)
34 "Collective civic nouns in 19th century state rights political discourse"
35
36 Pattern across all 10: Your SAE is dominated by two topics – Valkyria Chronicles and Little Rock Arsenal – confirming the c
37
38 ""
```

'\nNeuron 2659 (token: "1", sentence: Valkyria manga adaptations)\n"Capitalized poetic title words in Japanese media adaptation s"\nNeuron 1158 (token: ",", sentence: Valkyria Chronicles III development)\n"Comma punctuation in dense multi-clause game deve lopment prose"\nNeuron 3486 (token: "aspect", sentence: Little Rock Arsenal speech)\n"Formal abstract nouns in 19th century leg al authority language"\nNeuron 45 (token: ") ", sentence: Valkyria Chronicles III intro)\n"Closing parentheses in bilingual Japa nese-English title translations"\nNeuron 3549 (token: "charge", sentence: Little Rock Arsenal speech)\n"Words of authority and custody in formal governmental demand language"\nNeuron 2414 (token: "period", sentence: Valkyria Chronicles anime)\n"Time peri od references in entertainment media release scheduling language"\nNeuron 1289 (token: "mission", sentence: Valkyria Chronicles III development)\n"Military mission terminology in video game design and development context"\nNeuron 1778 (tok...

1 label

```
{'Capitalized poetic title words in Japanese media adaptations',
'Closing parentheses in bilingual Japanese-English title translations',
'Collective civic nouns in 19th century state rights political discourse',
'Comma punctuation in dense multi-clause game development prose',
'Definite article 'the' in formal governmental possession and authority claims',
'Formal abstract nouns in 19th century legal authority language',
'Military mission terminology in video game design and development context',
'Sales and commercial performance metrics in Japanese gaming charts',
'Time period references in entertainment media release scheduling language',
'Words of authority and custody in formal governmental demand language'}
```

```
1 _featlabel = pd.DataFrame(label)
2 _featlabel
```

0	Comma punctuation in dense multi-clause game d...
1	Formal abstract nouns in 19th century legal au...
2	Time period references in entertainment media ...
3	Capitalized poetic title words in Japanese med...
4	Collective civic nouns in 19th century state r...
5	Closing parentheses in bilingual Japanese-Engl...
6	Sales and commercial performance metrics in Ja...
7	Words of authority and custody in formal gover...
8	Definite article 'the' in formal governmental ...
9	Military mission terminology in video game des...

```
1 feature_one_interpretability['label'] = _featlabel
2 feature_one_interpretability.head()
```

	sentence index per neuron	feature indices per neuron	sentence	feature indices per neuron decoded	all_activ_neuron	label
0	30	2659	Two manga adaptations were produced , followi...	1	(3845, 8900, 9190, 9168, 9059, 9076, 9096, 921...	Comma punctuation in dense multi-clause game d...
1	13	1158	Concept work for Valkyria Chronicles III bega...	,	(9076, 9096, 3845, 9356, 9059, 2203, 13681, 12...	Formal abstract nouns in 19th century legal au...
2	41	3486	This movement is prompted by the feeling that...	aspect	(8573, 979, 985, 3845, 9096, 9076, 2203, 9356,...	Time period references in entertainment media ...
3	1	45	Senjō no Valkyria 3 : Unrecorded Chronicles (...	)	(9168, 9190, 8900, 9356, 9059, 9096, 9076, 220...	Capitalized poetic title words in Japanese med...

Next steps: [New interactive sheet](#)

```
1 _path = 'model/result/interpretations'
2
3 feature_one_interpretability.to_pickle(os.path.join(_path, 'full_feature_one_interpretability.pkl'))
```

```
1 from google.colab import files
2 import os
3
4
5 file_to_download = os.path.join('model/result/interpretations', 'feature_one_interpretability.pkl')
6
7
8 files.download(file_to_download)
```

```
1 files.download(os.path.join('model/result/interpretations', 'full_feature_one_interpretability.pkl'))
```

```
1 !zip -r model_results.zip /content/model

adding: content/model/ (stored 0%)
adding: content/model/result/ (stored 0%)
adding: content/model/result/interpretations/ (stored 0%)
adding: content/model/result/interpretations/full_feature_one_interpretability.pkl (deflated 53%)
adding: content/model/result/interpretations/feature_one_interpretability.pkl (deflated 52%)
adding: content/model/sae_layer14.pt (deflated 7%)
```

```
1 torch.save(sparsity.state_dict(), os.path.join('model', 'sae_layer14.pt'))
```

```
1 files.download('model_results.zip')
```

```
1 all_token_ids = torch.cat([
2     tokenizer(nb, return_tensors='pt', truncation=True, max_length=128)['input_ids']][0]
3     for nb in text
4 ])
```

```
1 """
2
3     This takes a sample row from hidden ( 18639, 3584 ), ( , 3584)
4     Gets top activating neuron
5     return top activating neuron for the given token
6
7     so we get a 2d vector (18639, 10), each row is a feature/token and each row with 10 columns containing top activating
8
9     td gives us the token indices for the activating neurons, we get what token get fired/activated, when the model sees a
10
11
12 """
13
```

```

14 # tv, td = hidden.topk(10, dim=-1)
15 # td.shape, tv.shape

```

```

1 """
2 Input sentence
3     ↓ tokenizer
4 (1, seq_len)          # token IDs, 1 sentence, seq_len tokens
5
6     ↓ model forward pass
7 (1, seq_len, 896)     # hook captures this – batch × tokens × hidden dim
8
9     ↓ squeeze + concat across 338 sentences
10 (18639, 896)         # all tokens stacked, each row is one token
11
12     ↓ DataLoader with batch_size=256
13 (256, 896)           # 256 tokens at a time fed to SAE
14
15     ↓ SAE encoder + ReLU
16 (256, 3584)          # each token now has 3584 sparse feature activations
17
18     ↓ SAE decoder
19 (256, 896)           # reconstructed back to original hidden dim
20
21 activations_9         # shape (18639, 896)
22 activations_9[0]      # shape (896,) – first token's full vector
23 activations_9[:, 0]   # shape (18639,) – dimension 0 across all tokens
24
25 hidden               # shape (18639, 3584)
26 hidden[:, 0]         # shape (18639,) – neuron 0's activation across all tokens ← this is what you used for interpretat
27 hidden[0]            # shape (3584,) – all neuron activations for first token
28 """

```